

Tephra

Refinement-Aware Mathematical Reasoning over Lean

Identity-preserving properties, behavioral contracts,
and observation-directed proofs

Central proposal. Extend the *elaborated mathematical type* of an object with proved properties and the type of a transformation with proved behavioral contracts. Use these contracts to infer the small steps a mathematician leaves implicit. Keep the original object, its chosen structures, and the strength of every observation fixed. Compile the resulting reasoning into ordinary Lean terms rather than importing mathematical theorem search into kernel conversion.

Prepared for Vladimir Reshetnikov

6 September 2026

Status: a technical design article with written mathematical proofs, an executed Python boundary model, and a small Lean reference encoding. The proposed language and elaborator have not been implemented; the supplied Lean encoding was not compiled in this environment.

Abstract

Lean can already express a positive real number, a smooth function, a torsion point, or a function satisfying a behavioral specification. The difficulty is not that these objects are inexpressible. It is that their properties, the operations they justify, and the observations they support must repeatedly be connected by explicit proof engineering. The two second-round syntheses establish the essential acceptance discipline: computations return values with proofs of exact, scoped specifications. This article proposes a third-round extension on the *consumer* side of that discipline: a refinement-aware elaborator that makes proved properties available as mathematical typing information, automatically assembles contract-preserving transformations, and records the proof of every implicit inference.

The design distinguishes object refinements, data-bearing structures, behavioral contracts, and observation relations. It gives a bidirectional elaboration calculus, explicit rules for contract variance and composition, a treatment of witness-dependent obligations, and a conservative interpretation into Lean. Two developments supplement the previously central analysis examples. A theorem about prefix-local linear transformations of streams explains precisely when finite observations can reverse a one-sided inverse. A restriction theorem for group actions explains the subtype and structure plumbing in the inspected Galois-representation code. The treatment of Leant uses its current length-contract and behavioral-selection interfaces rather than assuming that callback acceptance establishes behavior. It also addresses a subtle foundational limit: arbitrary polymorphism in classical Lean does not imply parametricity.

The article specifies a proof-carrying polynomial-checking boundary, connects it to guarded algebra and formal jets, evaluates genuine kernel extensions against an elaborator extension, and proposes a staged implementation with separate library, inference, and syntax measurements. Twenty-three executed Python tests check finite arithmetic and selected interface boundaries. They are not presented as formal verification of the language or of the general mathematical theorems.

Contents

1	The decision this round should make	3
2	What the inspected Lean code actually asks us to improve	4
3	A first look at the proposed language	6
4	What should be extended in Lean’s type system?	7
5	A small refinement elaboration calculus	9
6	Scope, dependency, and mathematical identity	11
7	Observations as typed interfaces, not automatic equalities	12
8	A worked behavioral theorem: prefix-local inverses	14
9	A second worked interface: restricting an action to torsion	16
10	The analysis control: differentiating an identity on an open set	18
11	Certified algebra: an implementation boundary with an actual denotation	19
12	Guarded algebra and a regular-branch jet type	21
13	Leant integration: from selected terms to proved behavior	23
14	Design alternatives for a genuine language or kernel extension	26
15	Elaborator architecture and implementation cuts	28
16	Executable evidence supplied with this article	30
17	Evaluation: separate mathematical abstraction from language benefit	32
18	Responses to the next-round implementation questions	33
19	Conclusion	34
A	A compact semantic interface	35
B	Source snapshots and reading coverage	35
C	Reproducing the delivered artifacts	36

1 The decision this round should make

1.1 From certified results to property-aware use

The previous round has already answered a substantial part of the problem. Its accepted computational object has the form

$$(y, p), \quad y : B(x), \quad p : \text{Spec}(x, y),$$

where the original input, context, and specification have been fixed. Equality, equality on a domain, an enclosure, a finite jet, a witness, and a complete solution predicate are not interchangeable outputs. Both synthesis reports make this point carefully, including the hypotheses required to pass from one guarantee to another [1, 2].

The next useful question is therefore not merely how another surface language should spell this pair. It is:

How can the system use the mathematical information already established about an object without asking the author to reconstruct the same interfaces at every operation?

A mathematician who introduces a prime $p \geq 5$ subsequently uses that p is positive, nonzero, odd, and suitable for constructing the field $\mathbb{Z}/p\mathbb{Z}$. A mathematician who introduces an additive action on a group expects it to preserve the subgroup of p -torsion points. A mathematician who has proved an identity on an open set differentiates that identity there, rather than repeatedly constructing neighborhood-equality evidence. These are not arbitrary omissions. They follow from specific, compositional mathematical contracts.

Tephra is a working name for a language layer that makes such contracts part of mathematical typing. Its essential extension is a second, proof-relevant elaboration judgment above Lean’s ordinary typing judgment. This is a real extension of the user-facing static semantics, but it need not be a new foundational logic.

1.2 What is inherited and what is proposed here

The architecture inherits fixed statements, dependent scopes, exact-target acceptance, theorem-backed methods, certified representations, service-independent replay, and honest result kinds from the syntheses. In particular, it adopts their corrections: complete solving requires both soundness and coverage; differentiation requires a genuinely local identity; collapsed bounds can imply equality; and a nonzero element need not be a unit [1, 2].

The additions developed here are more specific.

1. **Identity-preserving refinement elaboration.** A fact about an existing object enriches how that object may be used, without replacing it by a newly chosen representative or requiring repeated manual subtype projections.
2. **Behavioral typing of transformations.** Preconditions, postconditions, invariant preservation, locality, and commutation with observations are represented by ordinary propositions and consumed by explicit inference rules.
3. **Object-bound structure selection.** Data-bearing algebraic and topological structures are selected independently of the propagation of proposition-valued facts. The exact structure arguments belong to the meaning of a request.
4. **Two discriminating mathematical developments.** Prefix-local linear maps and invariant-subtype actions expose requirements that simple arithmetic examples do not: finite-dimensionality, observation coverage, closure, and coherence of induced structures.

5. **A current Leant integration path.** Provider laws become proof-linked registrations, while the existing distinction between receipt association and behavioral authority remains intact. Uniformity of polymorphic programs is a separately proved property, not an assumption about all Lean terms.

These are design contributions relative to the two syntheses, not claims to have invented refinement types, program contracts, or finite-dimensional linear algebra.

1.3 Evidence and scope of the review

Both supplied synthesis TeX sources were read through their conclusions and appendices. The code review is deliberately narrower than a repository audit: it examines the files registered in Appendix B, including their actual theorem statements and relevant proof bodies. The large generated instance-management preamble of the FLT solution file was inspected as infrastructure, not audited declaration by declaration. The entire FLT development was not rebuilt or independently validated.

The sources were retrieved as individual files from the repositories' current default branches. Their Git blob identities are retained. They are not represented as a single atomic multi-repository checkout. The syntheses' Lean timings and toolchain experiments are treated as *their* historical evidence; no such timings are relabeled as measurements made for this article. The new executable evidence is described separately in Section 16.

2 What the inspected Lean code actually asks us to improve

2.1 Binomial inversion: substantial abstraction already exists

`BinomialInversion.lean` does not simply repeat a long elementary calculation for every corollary. It proves an integer kernel identity, invokes a reusable lower-triangular transform calculus, and exposes additive-group and ring-valued versions. Its group-valued statement uses integer scalar multiplication and does not need a rational algebra or a field [4].

The remaining detailed work in the kernel identity includes interval reindexing, natural-subtraction bounds, conversion between ranges and closed intervals, a binomial-coefficient identity, and casts into $\mathbb{Z}$. These are four different phenomena. A shorter notation for sums does not itself prove that an index map covers the endpoint. A CAS over $\mathbb{Q}$ does not by itself justify an additive-group theorem. A single application of an already packaged inversion theorem should not be counted as a new language achievement.

The appropriate target is a proof such as: reindex the interval by $k = j + i$, use the product identity for binomial coefficients, and apply the alternating row sum. Each named operation should generate its precise bounds and transport obligations. The existing transform library must remain available unchanged to an ordinary Lean baseline.

2.2 Triangular kernels: the hidden behavior is prefix locality

`TriangularKernelInverse.lean` proves that one-sided orthogonality of lower-triangular kernels is two-sided. It cuts a kernel down to a finite square matrix, proves that matrix multiplication computes interval convolution, invokes the finite-matrix inverse theorem, and extracts an entry [5].

The human argument is brief because it silently carries two contracts. First, a lower-triangular operator's first N outputs depend only on the first N inputs. Second, the finite restriction acts on a finite free module, where a one-sided square-matrix inverse can be reversed over a commutative ring. Neither contract holds for an arbitrary endomorphism of an infinite object.

Section 8 makes both explicit and turns them into a reusable behavioral interface.

This is an additional development case, not a claimed sealed benchmark. Once it is used to design the system, it is no longer held-out evaluation material.

2.3 Autonomous derivatives: locality rather than a larger simplifier

`AutonomousIteratedDeriv.lean` already isolates a useful theorem. If $W' = \phi \circ W$ on an open set U , and a family G_n has the stated derivative and recurrence on the relevant range of W , then $D^n W = G_n \circ W$ on U . In the successor case, the proof constructs eventual equality near the point before taking derivatives [3].

This is a good example of information that should be consumed by the type layer. An identity available *on* U , together with openness and membership of the point, should satisfy a request for a local identity. An identity available only *at the point* should not. The proposed inference is the existing theorem-backed bridge, not a new axiom about differentiation.

2.4 Frey packages: mathematical refinements are already present

The inspected FLT definition stores nonzero integers a, b, c , a natural prime $p \geq 5$, a putative Fermat equation, a coprimality condition, and congruence normalizations in `FreyPackage`. It then derives positivity, nonzeroness and oddness of p , related coprimality statements, and nonvanishing products [6].

Thus it would be inaccurate to say that Lean cannot attach properties to mathematical objects. The package does exactly that. What remains is the repeated exposure and propagation of those properties, and the choice of the data-bearing structures that consume them.

The integral and rational Frey-curve definitions also contain divisions by 4 and 16. Their similar printed formulas do not license unrestricted transport between integer division and field division. Any inferred identification has to discharge exact-divisibility and cast hypotheses. The new interface must make these guards easier to obtain, not erase them.

2.5 Torsion actions: invariant-subtype lifting is repetitive but principled

`Def_FLTPrelim_GaloisRep.lean` constructs a Galois action on elliptic-curve points, proves that it commutes with integer scalar multiplication, proves that it preserves n -torsion, and restricts the action to the torsion subtype. Several action laws are then proved by reducing subtype equality to equality in the ambient point group [7].

The concise mathematical statement is that additive automorphisms preserve torsion. Section 9 develops the precise reusable interface. The contract includes the *chosen action*; it is not just a tag saying that some action exists on the carrier.

2.6 The end of the FLT argument is already short

The mathematical tail of `S_FreyPackage_no_frey_package.lean` obtains a nonzero level-two cusp form from the relevant modularity and irreducibility results, contradicts the vanishing theorem, and concludes by applying the irreducibility theorem. The final assembly is only a few lines. The file also contains a very large generated instance-disabling preamble [8].

This suggests an infrastructure experiment: can explicit, object-bound structure selection replace some global instance-management overhead? It does *not* show that the mathematics in the imported theorems has become dispensable, nor that a prose frontend will substantially shorten the final argument. The repository's proof-path document also distinguishes trace congruence from representation isomorphism and ordinary irreducibility from absolute irreducibility. A

language must retain those exact distinctions even if its displayed vocabulary is shorter [9].

3 A first look at the proposed language

3.1 Objects, facts, and operations

The intended surface is lightly structured mathematical notation, not an attempt to understand unrestricted prose. The following sketches use ASCII spellings for portability. They are proposed Tephra syntax, not commands that current Lean accepts.

```
context
  R : commutative ring
  F G : R-linear endomorphisms of Stream R
  F and G preserve prefixes

assume F o G = id
show   G o F = id
proof
  fix N : Nat
  observe the first N coordinates
  have F[N] o G[N] = id by observation of the assumption
  hence G[N] o F[N] = id by finite_free_inverse
  conclude by all_prefixes
```

The second line introduces actual linear-map structures; the third introduces proved behavioral properties. **observe** requests the registered finite restriction and its commuting theorem. **fix N** is a universal binder, not a loop running finitely many tests. **all_prefixes** is an extensionality theorem. The proof of the mathematical principle behind this sketch appears in Section 8.

The same object can acquire more information without changing identity:

```
given p : Nat where prime p and 5 <= p
have p is odd                      by prime_not_two
let k := ZMod p using its prime-field structure
```

The first declaration introduces a binder with these assumptions. A **construct** declaration with the same refinement would instead require a witness and its proofs. It is not permission to conjure a prime. The last line selects the canonical field structure through a named construction and records that selection. Proposition search may find the proof needed by that construction, but may not choose a different carrier or arithmetic structure.

3.2 A conditional calculation is still visibly conditional

```
calculate (x^2 - 1) / (x - 1)
  as an equality on {x : Real | x != 1}
  result x + 1
```

The domain is part of the requested relation. In a surrounding context already containing $x \neq 1$, a shorter calculation is allowed and its proof uses that fact. In an unrestricted context, the system cannot silently modify the theorem to assume $x \neq 1$. It may return the guarded result, request a proof of the guard, or propose an explicit case split.

Likewise:

```
solve x : K where a*x = b
  require a complete solution predicate
```

requires an answer handling $a = 0$ as well as $a \neq 0$. Finding b/a under the latter guard is useful intermediate work, but not completion of this request.

3.3 Conciseness comes from retained context, not hidden semantics

A mathematician should usually see the property at its introduction and the mathematical reason at its use, rather than every projection and instantiation. But a reader must still see a branch selection, a changed domain, finite precision, a completeness claim, or a newly assumed premise. The source and rendered views share the same typed record. Expanding a step shows the ordinary Lean proposition, the chosen theorem, its instantiated premises, and the resulting proof term or retained certificate.

For an already concise Lean term, the surface should allow a direct embedding:

```
by lean
  exact the_existing_corollary h
```

This is an escape into the same checking discipline, not an exception to it. A language that turns every one-line corollary into a mandatory page of structured prose has failed its purpose.

4 What should be extended in Lean's type system?

4.1 Expressivity is not the missing feature

Lean already has dependent function types, proposition-valued predicates, subtypes, structures with proof fields, and equality transport. A subtype $\{x : A \mid P(x)\}$ can encode a mathematical refinement directly. Its proof field does not become runtime data, and Lean's documented subtype representation is the same as the underlying value in compiled code [16, 17].

Nevertheless, the logical type $\{x : A \mid P(x)\}$ is not identical to A . Erasure of proof fields at runtime does not authorize erasing a predicate from the logic. In particular, identifying every refinement type with its carrier would identify an empty refinement with an inhabited carrier. Tephra never makes that identification.

The extension instead changes what the elaborator knows how to infer and insert. A property-aware type is a carrier together with an interface of available evidence. The carrier remains an ordinary Lean type; every admissible use is compiled to an ordinary Lean term.

4.2 Four strata that should not be collapsed

Stratum	Mathematical content	Example
Carrier	The kind of data	$x : \mathbb{R}, f : \mathbb{R} \rightarrow \mathbb{R}, s : \mathbb{N} \rightarrow R$
Chosen structure	Operations and their laws, often data-bearing	A field structure, scalar action, topology, algebra map
Refinement	A proposition about this object under these structures	$x > 0, f$ smooth on $U, s(0) = 1$
Behavioral contract	A proposition relating a transformation's inputs, outputs, or observations	Preserves torsion; depends only on a prefix; sends this jet precision to that precision

A behavioral contract is logically also a refinement, now of a function or other transformation. It deserves a separate interface because its variance, composition, and invocation generate characteristic obligations. An observation relation is not necessarily a refinement of the output alone: it is indexed by the original semantic object.

A structure may itself be proposition-valued, such as a property of a fixed operation. Conversely, some familiar type classes carry operations. The distinction is made by the actual declaration and arguments, not by a rule that all classes are data or all mathematical properties are classes. Lean's type-class mechanism already supports both forms [18].

4.3 Two representations of one refinement discipline

For a *first-class refined value*, use the ordinary interpretation

$$\llbracket A \mid P \rrbracket = \{x : \llbracket A \rrbracket \mid \llbracket P \rrbracket(x)\}.$$

For an existing local object, use an identity-preserving context representation:

$$x : A, \quad h_P : P(x), \quad h_Q : Q(x).$$

The surface may render this as “ $x : A$ satisfying P and Q .” No new x is constructed. The elaborator indexes h_P and h_Q by the same local variable, with their exact dependencies and structure arguments.

Crossing an API that genuinely expects a subtype inserts $\langle x, h_P \rangle$; returning to the carrier inserts the projection. Repeated refinement changes should not accumulate nested wrappers in local reasoning. This is an elaboration optimization and a stable identity policy, not a new definitional equality between distinct refinement types.

4.4 Why not put every fact in type-class search?

Turning every fact into a globally available instance invites uncontrolled recursive search and makes mathematical choices vulnerable to import order. More importantly, a proposition about a particular term and a selected domain has a different lifecycle from a canonical arithmetic structure.

Tephra uses a local proof index for facts and retains Lean's instance mechanisms for explicitly selected structures and compatible library interfaces. A rule can turn an available proof into a local instance when a theorem requires one, but only as an explicit bridge in the elaborated application. The system does not globally register every derived inequality or every alternative action on a carrier.

For example, two scalar actions on the same underlying additive group are not interchangeable just because their carriers print identically. Their dictionaries, algebra maps, and scalar-tower

proofs are part of the object-bound structure context. Proof irrelevance for two proofs of *one fixed proposition* does not identify two different pieces of mathematical data.

5 A small refinement elaboration calculus

5.1 Judgments and evidence

Let $\mathcal{E}$ be the Lean environment, and let Γ be an ordered Lean telescope containing types, data, structures, and proofs. An auxiliary index Φ points to some of the proof terms in this telescope or to closed proofs instantiated there. It is not a second source of logical assumptions.

Write

$$\mathcal{E}; \Gamma; \Phi \vdash e \Leftarrow A \mid P \rightsquigarrow (t, p)$$

when checking the surface expression e against a refinement produces

$$\mathcal{E}; \Gamma \vdash t : A, \quad \mathcal{E}; \Gamma \vdash p : P(t).$$

A synthesis judgment $e \Rightarrow A \rightsquigarrow t$ first finds the carrier and elaborated term. It may additionally return a finite collection of proved properties. It does not promise a strongest refinement or a principal type containing all mathematical consequences.

The difference between synthesis and checking is useful. Given a raw polynomial expression, the system can synthesize a polynomial carrier. Given a request for an invertible constant term, it checks the corresponding property. The property checker may use an indexed theorem, bounded automation, a user proof, or a certified computation. It may also leave an obligation open.

5.2 Introduction, forgetting, and subsumption

The basic rules are ordinary proof constructions.

$$\begin{aligned} t : A, p : P(t) &\Longrightarrow \langle t, p \rangle : A \mid P, \\ r : A \mid P &\Longrightarrow r.\text{value} : A, \\ r : A \mid P, h : \forall x, P(x) \rightarrow Q(x) &\Longrightarrow \langle r.\text{value}, h(r.\text{value})(r.\text{evidence}) \rangle : A \mid Q. \end{aligned}$$

The last rule is refinement subsumption. Its evidence is an implication proof, not a Boolean claim from a solver. For a fixed local object, it simply adds the derived proof of $Q(t)$ to the local proof index. Intersection introduction and elimination use conjunction introduction and projections.

This makes a type error more mathematically informative. A failed inverse request should say that it needs a proof of $\text{IsUnit}(a)$ for the selected ring, show the available nonzero or regularity facts, and identify the missing implication. It should not report merely that a coercion failed, nor assume that nonzeroness always supplies invertibility.

5.3 Dependent arrows and their two meanings

There are two interfaces that should not be confused.

For an *already existing total function* $f : A \rightarrow B$, define

$$\text{Contract}(P, Q, f) : \Longleftrightarrow \forall x : A, P(x) \rightarrow Q(x, f(x)).$$

Here the precondition restricts the guarantee, not the domain of the underlying total function. This suits imported Lean functions, including totalized arithmetic operations.

For a *new computation defined only on admissible inputs*, use a refined domain:

$$f : \prod_{x:A} (P(x) \rightarrow B(x)), \quad \forall x h, Q(x, f(x, h)).$$

Equivalently, one may pass x as a subtype element. This does not require an arbitrary value outside P . A synthesis request must choose one interface before search. The language must not silently extend a partial-domain computation to a total function by inventing default values.

Existence is also distinct from computation. From $\forall x, \exists y, Q(x, y)$ one may reason propositionally about witnesses. Producing a function in a data universe requires an appropriate constructive witness or an explicitly accepted choice construction. Lean’s restrictions on elimination from propositions are relevant here [17]. A proof of uniqueness makes a mathematical definition unambiguous, but does not automatically provide an executable algorithm.

5.4 Behavioral subtyping has a direction

Suppose f satisfies $\text{Contract}(P_1, Q_1, f)$. It can serve a requested contract (P_2, Q_2) if there are proofs

$$\forall x, P_2(x) \rightarrow P_1(x), \tag{1}$$

$$\forall x, y, P_2(x) \rightarrow Q_1(x, y) \rightarrow Q_2(x, y). \tag{2}$$

The caller’s assumptions must be sufficient for the implementation’s precondition, and the implementation’s postcondition must imply what the caller requests. Thus preconditions and postconditions vary in opposite directions. There is no useful universal numerical rank of contract strength.

Proposition 5.1 (Contract weakening). *Under (1)–(2), the following implication holds:*

$$\text{Contract}(P_1, Q_1, f) \implies \text{Contract}(P_2, Q_2, f).$$

Proof. Fix x with $P_2(x)$. Equation (1) gives $P_1(x)$, so the old contract gives $Q_1(x, f(x))$. Equation (2) gives $Q_2(x, f(x))$. The underlying function has not changed. $\square$

For dependent codomains or a changed representation, the adapter must additionally transport the result along the specified dependent map. An implication between postconditions alone cannot justify changing its carrier.

5.5 A compositional rule, including the missing bridge

Suppose

$$\text{Contract}(P, Q, f), \quad \text{Contract}(U, V, g).$$

To infer a contract for $g \circ f$, the system needs a bridge

$$\forall x, y, P(x) \wedge Q(x, y) \rightarrow U(y).$$

It then obtains

$$P(x) \rightarrow Q(x, f(x)) \wedge V(f(x), g(f(x))).$$

A desired postcondition $W(x, g(f(x)))$ follows from a separately supplied implication involving these facts. The result remains indexed by the actual intermediate value $f(x)$. Replacing that value by an arbitrary y chosen by another provider would invalidate the assembly.

The proof is direct: apply the first contract, use the bridge to establish the second precondition, apply the second contract, and compose the postconditions. This simple rule is the common form of guard propagation, range propagation, invariant preservation, and observation planning. It is implemented as ordinary implication application in the supplied Lean reference encoding.

5.6 A conservative elaboration claim, with its proper scope

Theorem 5.2 (Relative soundness of the small calculus). *Assume every primitive term and theorem used by an elaboration rule is well typed in $\mathcal{E}$, and every discharged obligation supplies a Lean proof in its recorded telescope. The rules above, together with ordinary dependent application, abstraction, products, equality transport, and theorem application, translate a successful refinement judgment into the two Lean judgments stated at the start of this section. No additional logical axiom is required by these rules.*

Proof. Induct on the finite elaboration derivation. Carrier synthesis is covered by the premise. Refinement introduction is subtype construction; forgetting is projection; subsumption applies its implication proof; intersection uses conjunction introduction or elimination. Application and abstraction use the corresponding dependent-function rules. Equality transport uses the equality eliminator with the actual supplied equality. A registered method contributes an application of its checked theorem to the recursively checked arguments. The composition rule is the proof just given. Every case constructs a term of the advertised type in the same telescope, so the induction closes. $\square$

This is a written theorem about a small specified calculus. It is not a machine-checked correctness proof of a future parser, registry, scheduler, or complete elaborator. In an implementation, ordinary Lean checking remains the final acceptance boundary even when the frontend is buggy.

6 Scope, dependency, and mathematical identity

6.1 A proof index is not a bag of attractive facts

An indexed fact should retain the elaborated proposition, its proof term, free-variable dependencies, selected structure arguments, source origin, and the operation that requested it. Retrieval may use a cheaper key, such as the proposition’s head and the identities of its principal objects. That key is only a search aid. Acceptance uses the actual proposition and proof.

For example, two displayed facts named “ $x \neq 0$ ” from different branches are not exchangeable. Neither are facts about $W(x)$ and about a freshly selected function $\widetilde{W}(x)$. The exact Lean term, not its pretty-printed spelling, identifies the object. Equality can support a transport, but that transport itself is a proof edge.

6.2 Dependent guards cannot be flattened

Consider an operation that first chooses y under guard $G(x)$ and then needs $H(x, y)$ to produce z . Its interface has the shape

$$\prod_{x:A} \prod_{h:G(x)} \sum_{y:B(x,h)} (Q(x, h, y) \times (H(x, h, y) \rightarrow C(x, h, y))),$$

with proposition-valued components represented appropriately in Lean. The essential point is the order of the telescope, not this particular packaging. H is instantiated at the retained y . It cannot be moved before y , borrowed from a sibling branch, or discharged using a result that itself depends on H .

A dependency graph makes the last restriction easy to explain. A guard node may depend only on already justified nodes in scope, or on subproofs checked without the result it authorizes. Cyclic proof search is allowed as an exploratory algorithm only if its accepted output is ultimately an ordinary well-founded proof term; a cycle of unproved receipts does not constitute evidence.

6.3 Freeze meaning, not every proof metavariable

The statement phase resolves binders, carrier types, universe parameters, meanings of notation, selected structures, domains, and any meaning-bearing choice the author has not explicitly delegated. Proof search is then allowed to instantiate proof arguments and local theorem parameters consistent with that fixed request.

This is not a ban on witness synthesis. A declaration such as “construct a polynomial satisfying this identity and degree bound” deliberately delegates the polynomial. Its value may be chosen by a provider, because the specification is fixed. By contrast, a provider may not replace an already quantified arbitrary solution of a functional equation by its preferred canonical solution. It needs a theorem relating those objects.

The operational implementation can distinguish rigid and flexible metavariables. Rigid metavariables affecting the statement must be solved before issuing a proof request. Flexible witness metavariables belong to explicitly declared construction obligations, and their assignments are committed transactionally with the evidence that validates them.

6.4 Reuse through checked context maps

Exact request identity is a useful cache fast path. More general reuse is justified by a context substitution $\sigma : \Gamma \rightarrow \Gamma'$ together with an entailment adapter for the requested guarantee. Applying σ to a proof produces evidence in the new context, provided all declarations and their dependencies are correctly mapped.

This explains the variance of useful reuse. A jet known through order N can answer a request through order $M \leq N$. Equality on a larger domain can answer equality on a smaller one. A theorem requiring *more* assumptions cannot be reused in a context proving fewer. A different axiom policy is a separate admissibility check, even if the proposition matches. These are the entailment-based reuse principles advocated by the first synthesis, now integrated with refinement checking [1].

6.5 Structure footprints

The meaning of $x + y$, $\sigma \cdot x$, or $f'(x)$ includes chosen operations and structures. A *structure footprint* records the explicit Lean dictionary and map arguments actually used in the elaborated expression. It is not a new logical object that a kernel has to trust.

In the FLT setting, a scalar tower contains particular maps between base fields and extension fields; an action contains particular maps on points. Tephra may generate local instances or explicit arguments from an object-bound selection, but an ambiguous competing selection is an elaboration conflict rather than an opportunity for search to change the theorem. The proposed experiment is to compare this discipline with the inspected generated instance preamble, not to assert beforehand that it will be faster.

7 Observations as typed interfaces, not automatic equalities

7.1 Exact presentations, observations, and relations

An exact presentation supplies data d with a validity predicate and a decoding theorem identifying it with a fixed object a . A deterministic observation supplies only $o(a)$, for some observation map $o : A \rightarrow B$. A general relation $\mathcal{R}(a, b)$ also covers information such as an interval enclosing a real number, where b need not determine a .

All three fit a result type indexed by the original object:

$$\text{Result}_{\mathcal{R}}(a) = \{b : B \mid \mathcal{R}(a, b)\}.$$

But their available operations differ. For two exact presentations decoding to a , their decodings agree. For two observations of a , agreement is available only for a common observation and compatible indices. There is no reason for unrelated representations to possess a global canonical normal form. This adopts the representation distinctions emphasized in the syntheses and the Facet and Vantage material they analyze [1, 2].

7.2 The commuting-square contract

For observations $o_A : A \rightarrow U$ and $o_B : B \rightarrow V$, a function $f : A \rightarrow B$ is implemented at the observation level by $\hat{f} : U \rightarrow V$ when

$$o_B \circ f = \hat{f} \circ o_A. \quad (3)$$

The proof of this equation is a behavioral contract. It says exactly which information is sufficient to compute the requested observation of $f(a)$.

Proposition 7.1 (Composition of observation contracts). *If $o_B \circ f = \hat{f} \circ o_A$ and $o_C \circ g = \hat{g} \circ o_B$, then*

$$o_C \circ g \circ f = \hat{g} \circ \hat{f} \circ o_A.$$

Proof. For each a , substitute the first equality into $o_C(g(f(a))) = \hat{g}(o_B(f(a)))$. □

The rule is trivial algebraically and important architecturally: the planner may compose two operations without inspecting either implementation once it has their commuting theorems. If a theorem is conditional, its guard is transported at the actual intermediate value as in Section 6.

7.3 Observation coverage is a separate theorem

A family $(o_i)_{i \in I}$ is jointly faithful if

$$(\forall i, o_i(a) = o_i(b)) \implies a = b.$$

Finite prefixes of a stream and all coefficients of a formal power series are examples. One finite prefix is not jointly faithful; neither is a finite collection of jets of bounded order. The rule `conclude by all_prefixes` therefore consumes a universally quantified proof and a registered joint-faithfulness theorem.

It is equally important to distinguish preservation from reflection. A ring homomorphism transports polynomial identities. Reflecting equality requires an injectivity theorem. Reflecting an existential statement such as ideal membership may require substantially more: $X \in (2X)$ holds over $\mathbb{Q}[X]$ but not over $\mathbb{Z}[X]$, although the coefficient inclusion is injective. The witness $1/2$ does not descend. The language records the actual reflected proposition, not a blanket “injective view” permission.

7.4 An observation-demand calculus

An operation can request the weakest *registered sufficient* input guarantee that supports its output. This is not a claim to compute a mathematical weakest precondition in general.

For formal series, write $f \equiv_N g$ when their coefficients agree at every index $k < N$. Typical theorem-backed rules are:

Operation	Sufficient information	Guaranteed output
Addition	$f \equiv_N \tilde{f}, g \equiv_M \tilde{g}$	$f + g \equiv_{\min(N,M)} \tilde{f} + \tilde{g}$
Multiplication	The same	$fg \equiv_{\min(N,M)} \tilde{f}\tilde{g}$
Differentiation	$f \equiv_{N+1} \tilde{f}$	$f' \equiv_N \tilde{f}'$
Inversion	$f \equiv_N \tilde{f}$, both constant terms units	$f^{-1} \equiv_N \tilde{f}^{-1}$
Restriction	$f \equiv_N \tilde{f}, M \leq N$	$f \equiv_M \tilde{f}$

The derivative rule follows from $[t^k]f' = (k+1)[t^{k+1}]f$. The inverse rule follows from $f^{-1} - \tilde{f}^{-1} = f^{-1}(\tilde{f} - f)\tilde{f}^{-1}$. Multiplication by a series of known positive order can yield a sharper bound, but that is another registered theorem, not a heuristic exception to the type system.

The backward planner can therefore request one more input coefficient before a derivative. The forward checker validates whatever order was actually obtained. A partial result is retained at its actual precision and cannot satisfy a stronger request merely because its polynomial looks plausible.

8 A worked behavioral theorem: prefix-local inverses

8.1 Why this example tests more than notation

The finite-matrix argument in `TriangularKernelInverse.lean` suggests an interface for transformations that respect finite observations. Its essence can be formulated independently of any particular kernel [5]. The following result is a reusable packaging of that argument, not a claim of a newly discovered algebraic phenomenon.

Let R be a unital commutative ring and let

$$\mathcal{S}_R = R^{\mathbb{N}}$$

be the R -module of streams. For $N \in \mathbb{N}$, let

$$\pi_N : \mathcal{S}_R \rightarrow R^{\{0, \dots, N-1\}}$$

be the prefix projection, and let ι_N extend a finite vector by zero outside the prefix. Both maps are R -linear and $\pi_N \iota_N = \text{id}$.

Definition 8.1 (Prefix locality). A map $F : \mathcal{S}_R \rightarrow \mathcal{S}_R$ is prefix-local if, for every N and all streams s, t ,

$$\pi_N(s) = \pi_N(t) \implies \pi_N(F(s)) = \pi_N(F(t)).$$

This is a relational behavioral property: it compares two executions at two inputs. It cannot be expressed adequately by a postcondition saying only that $F(s)$ is a stream. Continuity for the product topology is not an interchangeable condition; an output may depend continuously on a later input coordinate.

8.2 Constructing the finite restriction

For a prefix-local map define

$$F_N = \pi_N \circ F \circ \iota_N.$$

Lemma 8.2 (Restriction contract). *For every prefix-local F ,*

$$\pi_N \circ F = F_N \circ \pi_N. \tag{4}$$

If F is R -linear, then F_N is R -linear. If F, G are prefix-local, then $F \circ G$ is prefix-local and

$$(F \circ G)_N = F_N \circ G_N.$$

Proof. The streams s and $\iota_N \pi_N(s)$ have the same first N coordinates. Prefix locality gives

$$\pi_N F(s) = \pi_N F(\iota_N \pi_N(s)) = F_N(\pi_N(s)),$$

which proves (4). Linearity follows by composing the three linear maps defining F_N . If s, t agree on the prefix, locality of G gives prefix agreement of $G(s), G(t)$, and locality of F gives prefix agreement after the composition. Finally, for a finite vector u ,

$$\begin{aligned} (F \circ G)_N(u) &= \pi_N F(G(\iota_N u)) \\ &= F_N(\pi_N G(\iota_N u)) \\ &= F_N(G_N(u)). \end{aligned}$$

□

The zero extension is a convenient construction, not a semantic choice that affects the answer: any extension with the same prefix gives the same observed result by locality. Its invariance theorem is the contract that allows the extension to remain hidden from the mathematical text.

8.3 The finite algebraic step

Lemma 8.3 (One-sided inverses of finite square matrices). *For $A, B \in M_N(R)$, $AB = I$ implies $BA = I$.*

Proof. The case $N = 0$ is immediate. For $N > 0$, taking determinants gives $\det(A) \det(B) = 1$, so $\det(A)$ is a unit. The adjugate identity constructs a two-sided inverse $C = \det(B) \operatorname{adj}(A)$ of A . Then $B = (CA)B = C(AB) = C$, hence $BA = CA = I$. □

No division in a field was used. The relevant ring property is stable finiteness, supplied here by commutativity through the determinant argument. A library can later generalize the contract to other coefficient rings with a proved stable-finiteness interface.

8.4 The infinite conclusion

Theorem 8.4 (Reversing a prefix-local linear inverse). *Let $F, G : \mathcal{S}_R \rightarrow \mathcal{S}_R$ be R -linear and prefix-local. If $F \circ G = \operatorname{id}$, then $G \circ F = \operatorname{id}$.*

Proof. For every N , the restriction contract gives

$$F_N \circ G_N = (F \circ G)_N = \operatorname{id}.$$

The finite prefix module is finite free of rank N . Representing its linear maps by square matrices and applying the preceding lemma yields $G_N \circ F_N = \operatorname{id}$. Thus for every stream s ,

$$\pi_N(G(F(s))) = G_N(F_N(\pi_N(s))) = \pi_N(s).$$

This holds for every N . For any coordinate k , choose $N = k + 1$ and read the k th coordinate. We obtain $G(F(s))(k) = s(k)$ for all k , and function extensionality gives the result. □

The proof uses an arbitrary natural number N . Checking the identity for $N = 1, \dots, 1000$ would not prove the theorem. This is precisely the distinction that the `observe/all_prefixes` interface must preserve.

8.5 Recovering triangular-kernel inversion

For a lower-triangular kernel K , define

$$(T_K s)(n) = \sum_{k=0}^n K(n, k) s(k).$$

This is linear, and it is prefix-local because the output coordinate $n < N$ only uses $k \leq n < N$. Its finite matrix has entry $K(i, j)$ for $j \leq i$ and zero otherwise. The product-entry theorem identifies matrix multiplication with

$$(K \star L)(n, j) = \sum_{k=j}^n K(n, k) L(k, j).$$

Therefore a universally quantified identity $K \star L = \delta$ yields the reverse identity $L \star K = \delta$ through the finite restrictions. The source file spells out the finite-index and interval adapters; Tephra would obtain them from the registered restriction and product-entry contracts.

For sequences valued in an arbitrary additive commutative group M , do *not* apply Theorem 8.4 directly by pretending that M^N is finite free over $\mathbb{Z}$. Instead prove the integer kernel identity and then apply its scalar-action theorem to M . This preserves the generality of the existing binomial inversion theorem. The type layer is valuable precisely because it distinguishes these two routes.

8.6 Negative neighbors that explain the hypotheses

Define the stream shifts

$$(Ls)(n) = s(n+1), \quad (Rs)(0) = 0, \quad (Rs)(n+1) = s(n).$$

Then $L \circ R = \text{id}$, while $R \circ L$ erases the zeroth coordinate. Both are linear. But L is not prefix-local: two streams can agree at coordinate 0 and disagree at coordinate 1, so their images under L disagree at coordinate 0. A refinement checker must refuse the finite-restriction interface for this map.

Prefix locality alone is also insufficient over an infinite carrier. On integer streams, the coordinatewise maps $s(n) \mapsto 2s(n)$ and $s(n) \mapsto \lfloor s(n)/2 \rfloor$ are prefix-local and form a one-sided inverse pair, but not a two-sided one. The halving map is not $\mathbb{Z}$ -linear. These examples separate the two behavioral/structural premises and give a stronger test than a type mismatch manufactured by misspelling a theorem name.

9 A second worked interface: restricting an action to torsion

9.1 The generic invariant-subtype theorem

Let A be a type, let a monoid G act on A by a fixed action ρ , and let $P : A \rightarrow \text{Prop}$ satisfy

$$\forall g, a, \quad P(a) \implies P(\rho(g, a)). \tag{5}$$

Define

$$\rho_P(g, \langle a, h \rangle) = \langle \rho(g, a), \text{proof from (5)} \rangle.$$

Proposition 9.1 (Restriction of an action). *The operation ρ_P is an action on $\{a : A \mid P(a)\}$, and its projection to A commutes with ρ .*

Proof. Closure is exactly (5). Projecting the identity-action equation gives $\rho(1, a) = a$. Projecting the composition-action equation gives $\rho(gh, a) = \rho(g, \rho(h, a))$. These are the ambient action laws. Equality of subtype values follows from equality of their underlying values; their proof components do not create additional choices. The commuting statement is the defining equation of ρ_P after projection. $\square$

The theorem induces an action, not every algebraic structure one might desire on the subtype. To induce an additive group structure, P must describe an additive subgroup, with the corresponding closure properties. An invariant subset that omits zero is not an additive subgroup merely because the action preserves it.

9.2 The torsion instance

Now let A be an additive commutative group and assume the selected action is by additive automorphisms. Set

$$A[n] = \{a : A \mid n \cdot a = 0\}.$$

For $a \in A[n]$,

$$n \cdot \rho(g, a) = \rho(g, n \cdot a) = \rho(g, 0) = 0.$$

The first equality is preservation of repeated addition. This establishes the guard needed by the generic restriction theorem. Moreover, $A[n]$ is an additive subgroup: it contains zero, and the equations for sums and negatives follow from distributivity of n -fold addition in the commutative group.

The inspected Galois-representation file performs these constructions for elliptic-curve points and then supplies a $\mathbb{Z}/n\mathbb{Z}$ -module structure on the torsion group [7]. Its proof of invariance rewrites torsion membership, commutes the action past scalar multiplication, and uses the zero-action law. These are exactly the premises above.

A possible surface is:

```
context
  A : additive commutative group
  rho : action of G on A by additive automorphisms
  n : Nat

let V := {a : A | n*a = 0}
derive the subgroup structure on V by torsion_subgroup
derive action rho on V
  by restrict_action preserving V because additive_maps_preserve_torsion
```

The last declaration preserves the identity of ρ rather than searching for any action on V . The generated action laws are accepted through a generic theorem, not generated as unsupported facts.

9.3 What the interface must not infer

For a prime p , the selected prime-field structure on $\mathbb{Z}/p\mathbb{Z}$ is another justified construction. It does not follow from a merely positive modulus. Nor does the existence of the torsion module establish that it has dimension two, that it is nontrivial, that the representation is irreducible, or that it is absolutely irreducible. Each is a separate predicate with its own mathematical proof.

The source definition of irreducibility quantifies over stable submodules and includes a nontriviality condition. A shorter displayed property may refer to that definition, but cannot silently drop nontriviality or replace the quantifier over submodules by a search through a few candidates.

Similarly, the base field and algebraic-closure arguments in the action cannot be inferred by textual name matching. They are part of the structure footprint.

9.4 A meaningful FLT-facing result

The deliverable is not a replacement proof of FLT. It is a library interface that can turn repeated invariant-subtype constructions into declarations with automatically reconstructed evidence. A useful measurement counts subtype coercions, closure lemmas, repeated action-law proofs, and structure-resolution work separately from the mathematical theorems imported by the final contradiction.

The short final contradiction should stay short. It can either remain as Lean source or be displayed as: the Frey package gives the relevant irreducible representation; modularity and level lowering give a nonzero level-two weight-two cusp form; that space is zero. The exact instantiated theorems must remain inspectable, especially where the repository uses a specialized notion such as trace congruence rather than representation isomorphism [8, 9].

10 The analysis control: differentiating an identity on an open set

Let $U \subseteq \mathbb{R}$ be open. Suppose that for every $x \in U$,

$$W'(x) = \phi(W(x)),$$

in the precise sense of an available derivative witness. Let $G_n, G'_n : \mathbb{R} \rightarrow \mathbb{R}$ satisfy

$$\begin{aligned} G_0(W(x)) &= W(x), \\ G'_n &\text{ is the derivative of } G_n \text{ at } W(x), \\ G_{n+1}(W(x)) &= G'_n(W(x))\phi(W(x)). \end{aligned}$$

The middle line names $G'_n(W(x))$ as the derivative value; it does not assume that the entire function G'_n is itself differentiable. These are the range-local hypotheses of the inspected theorem [3].

Proposition 10.1. *For all $n \in \mathbb{N}$ and all $x \in U$,*

$$D^n W(x) = G_n(W(x)).$$

Proof. Induct on n . The zeroth case is the initial identity. For the successor case, fix $x \in U$. The induction hypothesis holds throughout U , and openness implies that it holds on a neighborhood of x . Thus $D^n W$ and $G_n \circ W$ have the same derivative at x . The chain rule gives derivative value $G'_n(W(x))\phi(W(x))$, and the recurrence identifies this value with $G_{n+1}(W(x))$. $\square$

The author-facing successor step can be the familiar calculation

$$D^{n+1}W = D(G_n \circ W) = (G'_n \circ W)(\phi \circ W) = G_{n+1} \circ W \quad \text{on } U.$$

The elaborator obtains eventual equality at a particular point from the first line's domain contract. It does not need a special analytic kernel rule. Nor does it need to strengthen the range-local hypotheses to global smoothness of every G_n .

A decisive negative example is $f(t) = t$ and $g(t) = 0$ at $t = 0$. Their values agree there; their derivatives do not. Therefore the available proposition $f(0) = g(0)$ cannot inhabit the input interface for differentiation of an identity. The error should identify the required local relation rather than attempt an unrelated proof that changes the mathematical plan.

11 Certified algebra: an implementation boundary with an actual denotation

11.1 The minimal polynomial language

The second-round reports correctly ask for a checker soundness theorem, but several of their experiments stop at a Boolean coefficient check [1, 2]. Here is the full mathematical boundary that a first implementation should realize.

Fix a number d of variables. A monomial is $\alpha \in \mathbb{N}^d$, and an executable integer polynomial is a finite map

$$p : \mathbb{N}^d \longrightarrow \mathbb{Z}$$

with finite support. Its canonical encoding sorts exponent vectors, combines duplicate monomials, and omits zero coefficients. For a commutative ring R and an exact valuation $v : \{0, \dots, d-1\} \rightarrow R$, define

$$\text{ev}_v(p) = \sum_{\alpha \in \text{supp}(p)} (p_\alpha : R) \prod_{j < d} v_j^{\alpha_j}.$$

The variable count, exponent nonnegativity, coefficient domain, and canonical encoding are checked before computation. In production, resource bounds on degree, support, integer bit sizes, and certificate length are additional admission checks; they are not hypotheses of the mathematical identity.

Addition merges coefficients, negation negates them, and multiplication is finite convolution:

$$(pq)_\gamma = \sum_{\alpha + \beta = \gamma} p_\alpha q_\beta.$$

There is no semantic dependence on the ordering chosen for the finite encoding.

11.2 The denotation lemmas

Lemma 11.1 (Evaluation respects the executable operations). *For every commutative ring R , valuation v , and executable polynomials p, q ,*

$$\text{ev}_v(p + q) = \text{ev}_v(p) + \text{ev}_v(q), \quad \text{ev}_v(-p) = -\text{ev}_v(p), \quad \text{ev}_v(pq) = \text{ev}_v(p) \text{ev}_v(q).$$

The zero polynomial evaluates to zero, and normalizing a finite coefficient list preserves its evaluation.

Proof. For addition, regard both maps as functions on the finite union of their supports and distribute the sum over the coefficient addition. Missing coefficients are zero and contribute nothing. Negation follows by distributing the finite sum over coefficient negation.

For multiplication, expand the product of the two finite sums. A term indexed by (α, β) has value

$$(p_\alpha q_\beta : R) \prod_{j < d} v_j^{\alpha_j + \beta_j},$$

by commutativity and the natural-power addition law. Regroup this finite double sum by $\gamma = \alpha + \beta$. Its coefficient is exactly the convolution above. This proves the product identity. Zero coefficients contribute zero, duplicate monomials combine by distributivity, and a permutation of a finite sum leaves it unchanged. These facts prove normalization invariance and the zero assertion. $\square$

These are the lemmas that connect fast coefficient data to semantic ring expressions. A Lean implementation may map first into Mathlib’s polynomial types and then evaluate, or prove the displayed evaluation identities directly. Either route must be completed and checked; a fast Boolean test alone is not enough.

11.3 An ideal-membership certificate and its full theorem

For target p , premises $f_1, \dots, f_m$, and candidate multipliers $q_1, \dots, q_m$, define the checker to accept exactly when the arities agree and canonical coefficient arithmetic verifies

$$p = \sum_{i=1}^m q_i f_i. \quad (6)$$

Unequal lists are rejected before any use of `zip`. An implementation that silently drops an unmatched element does not implement this interface.

Theorem 11.2 (Soundness of the polynomial certificate). *If the checker accepts, then for every commutative ring R and valuation v ,*

$$(\forall i, \text{ev}_v(f_i) = 0) \implies \text{ev}_v(p) = 0.$$

Proof. Acceptance gives the exact coefficient identity (6). Applying evaluation and the preceding lemma yields

$$\text{ev}_v(p) = \sum_{i=1}^m \text{ev}_v(q_i) \text{ev}_v(f_i).$$

Under the premise each summand is zero, so the finite sum is zero. $\square$

This proof does not assume that R is a domain or a field. The certificate is a sufficient positive witness, not a complete decision procedure for ideal membership. Rejection says that this certificate failed. It says nothing by itself about whether another certificate exists, or whether the original goal is false.

11.4 Reification is a separate, source-specific obligation

Suppose the original Lean goal concerns expressions $e, e_1, \dots, e_m$. The adapter must establish

$$e = \text{ev}_v(p), \quad e_i = \text{ev}_v(f_i)$$

with the exact selected operations and valuation. Only then do the hypotheses $e_i = 0$ and the checker theorem imply $e = 0$.

Opaque atoms are permitted. A term such as $\sin x$ can become a variable in polynomial reasoning if its exact Lean expression is retained in the valuation. Repeated occurrences must map consistently; two different terms with the same display name cannot be merged. Treating an inverse as an opaque atom is also possible, but this supplies no inverse law unless such a law is separately proved with its guard.

The adapter must not reify a noncommutative multiplication as commutative, or import rational coefficients into a ring that has no selected rational-algebra structure. An equality after a coefficient extension is not automatically a certificate in the original coefficient domain. These are type and denotation obligations, not formatting details.

11.5 The execution link

Let $\text{check}(q, c)$ be the actual Lean checker and let its soundness theorem be proved. An untrusted process returning the bytes `true` does not produce a proof of $\text{check}(q, c) = \text{true}$. The strict path must establish that equation by kernel reduction, by an ordinary reconstructed proof, or by another justified checking mechanism. Then it can apply the soundness theorem.

Current Lean documentation distinguishes native evaluation from kernel-justified computation and explains the change to dedicated per-computation axioms from Lean 4.29 onward [19]. Consequently, the acceptance policy records transitive axiom dependencies rather than blacklisting a historical constant name. A term may be well typed relative to additional computation axioms yet fail the project's strict policy.

The proposed initial implementation uses kernel-checked proof production or a kernel-reduced checker. An accelerated mode is a separate explicit policy. No size threshold is prescribed from the syntheses' special-family timings. The crossover must be measured on the actual checker, coefficient sizes, and multivariate workload.

11.6 Existing automation is the control, not a competitor to ignore

Lean's documented `grobner` tactic handles polynomial-equation goals over commutative (semi)rings as a restricted frontend to `grind`; `grind` already propagates facts and constructs ordinary Lean proof terms [21, 22, 23]. These capabilities should be reused where they fit.

Tephra's adapter can initially retain an accepted proof term from such a tactic. A separate coefficient-certificate format is valuable only if it brings reproducibility, checking cost, or producer independence that justifies its implementation. This inspection does not establish a stable standalone certificate-export interface for `grobner`; that interface must be investigated or added rather than assumed. Failure under configured search limits is not a negative mathematical result.

12 Guarded algebra and a regular-branch jet type

12.1 Complete symbolic solving need not decide parameter equality

Over a field K , the solution set of $ax = b$ has the familiar specification

$$S(a, b) = \begin{cases} \{b/a\}, & a \neq 0, \\ K, & a = 0 \text{ and } b = 0, \\ \emptyset, & a = 0 \text{ and } b \neq 0. \end{cases}$$

The proof is elementary. In the first case multiply by a^{-1} . In the second case every x solves $0x = 0$. In the third case no x solves $0x = b$.

The output may remain a symbolic predicate with these cases. It is not an executable decision procedure for equality of arbitrary real parameters. A logical case distinction can use the project's classical reasoning policy without pretending that the program can compute which branch an arbitrary real inhabits. Completeness is a theorem about the represented solution predicate; operational branch selection is a different contract.

The pair of proof fields should be

$$\forall x, x \in S \rightarrow ax = b, \quad \forall x, ax = b \rightarrow x \in S.$$

A witness plus coverage is insufficient if the proposed set also contains false solutions. For example, $S = \{0, 1\}$ covers every solution of $x = 0$ but is not sound.

12.2 Totalized operations versus partial expressions

For ordinary Lean expressions, retain the imported total-operation semantics. Under the usual totalized field division, $(x^2 - 1)/(x - 1)$ is 0 at $x = 1$, whereas $x + 1 = 2$. Thus the displayed simplification is not a global identity of total functions.

An opt-in partial-expression type instead records a domain D and a value function defined on that domain. Equality of partial expressions should specify whether domains must coincide or whether only agreement on a named subdomain is claimed. Removing a pole produces either a guarded equality or a new extension object with an agreement theorem. It does not mutate the original domain silently.

The default for importing existing Lean code is the total interpretation. A worksheet may explicitly choose a partial-expression package. This choice is visible in the statement and in the type of its expressions, not an ambient convention that a CAS may silently change.

12.3 Regularity belongs to the branch, not to an attractive formula

Let R be a commutative ring and let

$$F(Y) \in R[[t]][Y].$$

Write $\bar{F}(Y) \in R[Y]$ for reduction of the coefficients at $t = 0$. Fix an existing series Δ with $F(\Delta) = 0$ and $\Delta(0) = d$.

A *regular-branch refinement* of this object stores the exact equation, the selected constant term d , the proof that Δ satisfies both, and a proof that $\bar{F}'(d)$ is a unit. It does not select Δ anew. This is the property-aware form of the residual theorem emphasized in the second-round reports [1, 2].

Theorem 12.1 (Residual certificate for the selected branch). *Assume $\bar{F}'(d)$ is a unit. Let $N \geq 1$ and let $J \in R[[t]]$ satisfy $J(0) = d$ and $F(J) \in (t^N)$. Then*

$$J - \Delta \in (t^N), \quad \text{equivalently } J \equiv_N \Delta.$$

Proof. Writing $F(Y) = \sum_{k=0}^r a_k Y^k$, the finite geometric-difference identity gives

$$F(J) - F(\Delta) = (J - \Delta)U,$$

where

$$U = \sum_{k=1}^r a_k \sum_{i=0}^{k-1} J^{k-1-i} \Delta^i.$$

Because $J(0) = \Delta(0) = d$, the constant term of U is

$$U(0) = \sum_{k=1}^r a_k(0) k d^{k-1} = \bar{F}'(d).$$

A formal series with a unit constant term is a unit: its inverse is constructed coefficient by coefficient, starting with the inverse of the constant term and solving each subsequent coefficient equation by multiplication by that inverse. Therefore U is invertible. Since $F(\Delta) = 0$ and $F(J) \in (t^N)$, multiplying the displayed factorization by U^{-1} gives $J - \Delta \in (t^N)$. $\square$

The reasoning needs neither a field nor a choice of a canonical solution. It validates a computed observation of the specific Δ already in the context.

12.4 What the type-directed request looks like

```
given Delta : RegularBranch F at d
request J : Jet 6 of Delta
  by residual_certificate
```

The phrase `RegularBranch` is shorthand for the explicit refinement just described. A candidate J must match d and make its residual vanish below 6. The retained theorem then identifies its coefficients with those of Δ . Requesting a derivative jet of order 5 can reuse this result; requesting order 6 requires one more input coefficient.

For the Catalan branch $C = 1 + tC^2$, the integer polynomial

$$J = 1 + t + 2t^2 + 5t^3 + 14t^4 + 42t^5$$

has residual $J - 1 - tJ^2$ with zero coefficients below degree 6 and coefficient -132 at degree 6. The companion checks this finite arithmetic exactly. The regular-branch theorem supplies the interpretation of that arithmetic once an existing Catalan series and its equation are present. The polynomial itself is not an exact solution of the full equation.

Two negative neighbors are essential. Over $\mathbb{Z}/4\mathbb{Z}$, take $F(Y) = 2Y$, $\Delta = 0$, and $J = 2t$. The residual vanishes, and the derivative 2 is nonzero, but $J \not\equiv_2 \Delta$: nonzero cannot replace unit. Over $\mathbb{Q}$, $F(Y) = Y^2 - 1$ has regular roots 1 and -1 ; zero residual does not identify the negative root with a separately selected positive root. The common constant term is a premise.

12.5 Three distinct algebraic capabilities

Cancellation from $ax = ay$ needs regularity of a as a multiplication map. Forming a^{-1} with inverse laws needs a unit. Deducing $p = 0$ from $p^k = 0$ needs an appropriate nilpotent-freeness hypothesis, such as reducedness, and $k > 0$. These are different predicates and different theorem interfaces.

The integer 2 is regular but not a unit in $\mathbb{Z}$. The element 2 of $\mathbb{Z}/4\mathbb{Z}$ is nonzero but not regular. A field-specific solver may use equivalences between some of these properties only when its field structure is part of the fixed context. Property-aware typing should expose these distinctions more reliably than a menu of broadly named “algebra” tactics.

13 Leant integration: from selected terms to proved behavior

13.1 What the current source already separates

The current `Verification.hs` describes `Verified candidate` as a receipt that a verification callback accepted a candidate. It explicitly does not claim to be behavioral evidence, a solver certificate, or a kernel proof [10]. The current behavioral-selection interface similarly seals a total, ordered partition of the exact input receipts in a fresh epoch; its association guarantees are not proofs of behavior [12].

The length-contract vocabulary now records exact family and constructor names, argument roles, candidate-case policies, provider transfer laws, and scalar or binary-product result domains. These values are passive assertions until the integration machinery associates them with the appropriate provenance [13]. The adapter distinguishes this association from a source-level Lean claim and warns that a replayed abstract model has only the authority specified by its receipt [14].

This is a useful starting point, not an omission to rediscover. Tephra should add a proof-linked

registration layer around those contracts rather than rename their current wrappers and claim the result is verified mathematics.

13.2 Proof-linked provider laws

For a provider such as list append, the useful behavioral law is

$$\text{length}(xs ++ ys) = \text{length}(xs) + \text{length}(ys).$$

A registration should pair the exact provider constant and its instantiated type with a Lean theorem establishing the transfer law, the positions of the data arguments, the observation map, and any premises. The provider's name alone is not evidence of its meaning.

The existing passive law format can remain useful for exploratory ranking. There are two ways to promote a suggestion to a justified decision. One is to validate every provider law and the abstract interpreter's semantic theorem, then reconstruct or check the logical result. The other is to treat the analysis only as a generator of candidates or counterexamples and verify the proposed consequence directly against the original Lean term. The latter allows incremental integration without pretending that all existing law metadata is already certified.

A supported length fragment can use constants and nonnegative linear combinations of input lengths, with explicit case splits when the admitted constructors require them. An operation such as reverse has the identity transfer law on lengths. This does not imply that reverse and identity have the same behavior: they differ on the input $[1, 2]$ despite identical lengths. The target specification decides whether length alone is sufficient.

13.3 Positive and negative abstract reasoning need different bridges

Let Bad_{src} describe violations by an actual candidate at actual inputs, and let Bad_{abs} describe abstract violations. To prove that no source violation exists from the absence of abstract violations, a sufficient bridge is

$$\text{Bad}_{\text{src}} \implies \text{Bad}_{\text{abs}}.$$

To refute a source specification using an abstract violation, one needs a way to obtain an actual source violation from that particular abstract witness. An overapproximation of possible behavior generally supplies the first direction, not the second.

For a concrete function and a universally quantified specification, one *realized*, *checked* counterexample is enough:

$$a : A, \quad h : \neg Q(a, c(a)) \implies \neg \forall x : A, Q(x, c(x)).$$

This is entirely different from promoting a collection of passing tests to a universal theorem. It also refutes only the specific candidate c , not the existence of some other candidate satisfying the specification.

An implementation should therefore distinguish at least: a search hint; callback-accepted syntax; a proved contract for the exact candidate; a realized counterexample to that candidate; and an inconclusive outcome. These are kinds of evidence and operational status, not points on a single confidence scale.

13.4 A missing behavioral premise: observation realizability

Length observations make the issue particularly sharp.

Proposition 13.1 (The image of list length). *For a type A and $n \in \mathbb{N}$,*

$$(\exists xs : \text{List}(A), \text{length}(xs) = n) \iff n = 0 \vee \text{Nonempty}(A).$$

Proof. If a list of length n exists and $n \neq 0$, the list is nonempty, so its head witnesses nonemptiness of A . Conversely, the empty list realizes $n = 0$. Given an element $a : A$, the list containing n copies of a realizes any n . $\square$

This statement is propositional. An executable realizer for positive length needs an actual element of A , not merely an unexamined assumption about a constructor's name. A propositional existence proof can be sufficient for refuting another proposition without supplying executable extraction of a witness.

Consider the candidate

$$c : \text{List}(\text{Empty}) \rightarrow \text{List}(\text{Empty}), \quad c(xs) = [].$$

It *does* preserve length, because every list over the empty type is empty. An abstract length solver permitted to choose $n = 1$ would find the apparent violation $0 \neq 1$. That witness cannot be realized by a source input. Hard rejection on that basis would be wrong.

For a target quantified over all element types, a counterexample may instead instantiate the type with a known inhabited type, if that instantiation is allowed by the original quantifier scope. It may not replace a fixed element type by a convenient one. The exact family, type arguments, and binder positions already retained by Leant are therefore semantically important [11, 13].

The proposed observation interface has an optional *realization contract*: a theorem characterizing which abstract observations are attainable, and a witness construction where appropriate. This is not needed for every conservative proof-oriented abstraction. It is needed whenever an abstract witness is promoted to a source counterexample.

13.5 Polymorphism is not an implicit behavioral axiom

The type

$$\forall A : \text{Type}, \quad \text{List}(A) \rightarrow \text{List}(A)$$

does not state length preservation, order preservation, or even the usual naturality theorem for every Lean inhabitant. Lean's own axiom documentation gives a noncomputable polymorphic function that tests whether its element type is $\mathbb{N}$ and returns either the empty list or its input. Such type-sensitive behavior contradicts a blanket parametricity principle [20].

Consequently, Tephra should never infer a free theorem merely because a type is polymorphic. A useful additional refinement is a *uniformity contract*, for example

$$\text{map}(g)(f_A(xs)) = f_B(\text{map}(g)(xs))$$

for every $A, B, g : A \rightarrow B, xs$. This property can be supplied by a proof or generated by a proved relational translation of a restricted source fragment. An optional **uniform** declaration would mean that this check succeeded; it would not be a new axiom applicable to arbitrary imported Lean definitions.

A restricted syntactic fragment needs a precise admission rule for each primitive and imported provider. Proof-only classical reasoning may be harmless for some translations, whereas data-producing choice or type inspection may not be. That boundary must be established by the translation theorem, not guessed from whether a source file contains a particular keyword. Uniformity is a later package, not a prerequisite for the first refinement elaborator.

13.6 A full behavioral synthesis request

A richer list-splitting request might require

$$\begin{aligned} c(xs) &= (u, v), \\ u ++ v &= xs, \\ \text{length}(u) &= \min(k, \text{length}(xs)). \end{aligned}$$

Length reasoning can prune a candidate that returns too many elements, provided the counterexample is realized and checked. It cannot prove the concatenation equation. That part needs a sequence-level theorem or direct proof about the candidate. For a binary result, the order of the two product fields must be preserved; swapping them may preserve total length while violating the specification.

The final requested object is therefore

$$\{c : \text{List}(A) \rightarrow \text{List}(A) \times \text{List}(A) \mid \forall xs, \text{Spec}(k, xs, c(xs))\}.$$

Structural synthesis proposes its data component. Observational analysis narrows the search. Lean evidence establishes the complete predicate. No stage may substitute its own weaker target for this one.

13.7 Replay and negative search outcomes

The inspected replay module distinguishes reusing an environment, replaying an appended suffix, and replaying from the base after an incompatible history edit [15]. This is useful state management. A separate publication condition is that the exact displayed edit or retained term must be accepted at the exact original goal; an environment-replay plan alone does not establish it.

Search exhaustion, a timeout, missing solver output, or nonderivability in an abstract calculus remains an operational result. To conclude a Lean negation, a proof must be reconstructed at the original proposition. Even a mathematically complete search of a restricted term grammar does not show that no term exists outside that grammar. The property layer adds useful constraints but does not change this logical boundary.

14 Design alternatives for a genuine language or kernel extension

14.1 A property-implicit binder is the smallest useful extension

A declaration could mark an ordinary proof parameter as *property implicit*:

```
cancel(a, x, y)
  requires ha : Regular(a)
  from h : a*x = a*y
  returns x = y
```

At the declaration boundary, **requires** is an explicit hypothesis. At a call site, it requests a proof in the existing context. It never appends that hypothesis to the caller’s theorem automatically. Logically this is an ordinary dependent function parameter; operationally it uses the property resolver rather than unrestricted global instance search.

A second small extension is a *proof-directed coercion*: when an API expects a related refinement or dependent representation, the elaborator searches for a registered implication or equality theorem and inserts the corresponding subtype construction or equality transport. The resulting term

contains that evidence. This differs fundamentally from declaring the two types definitionally equal.

Both features can initially be implemented by Lean metaprogramming over existing types and proof parameters. The substantive language extension lies in the checking judgment, the controlled insertion policy, and the source-visible obligation model.

14.2 Which foundational changes would actually help?

Alternative	Potential benefit	Cost or limitation
Library-only refinements	Immediate compatibility; no new trusted mechanism	Authors still invoke most adapters explicitly
Refinement-aware elaborator	Implicit property propagation, guard inference, structure-aware requests	Requires a disciplined resolver, diagnostics, and evaluation
Native proof-bearing refinement former	More direct core notation or specialized storage	Must retain proof evidence; duplicates much of <code>Subtype</code> ; needs a measured benefit
General semantic subtyping in conversion	Very permissive apparent typing	Moves arbitrary implication proving into conversion; poor predictability and unclear practical boundary
Equality reflection	Propositional equalities can influence conversion directly	Changes the foundational checking problem; explicit transports already express the needed mathematics
Certified computation as an axiom-free primitive	Possible performance specialization	Requires a narrowly specified checked semantics and independent validation; not permission to trust arbitrary native answers

The recommended target is the second row. The third is a possible later optimization, not a prerequisite. A native refinement constructor must still carry or obtain evidence for its predicate; erasing the distinction in the logic would be unsound. If its semantics translate to subtypes and ordinary functions, its conservativity can be demonstrated by that translation before changing the kernel implementation.

Arbitrary semantic implication cannot be promised as a complete terminating inference service. A practical system chooses a bounded fragment or a bounded search strategy and leaves the remaining proof obligations explicit. The absence of a principal strongest refinement is not an error; mathematical libraries deliberately expose useful interfaces rather than every consequence of a definition.

14.3 Why equality transport is usually the right answer

Suppose the elaborator has $t : C(x)$ and a proof $p : x = y$, while an API expects $C(y)$. It may insert the ordinary equality eliminator applied to p and t . For an equality between types, it inserts the corresponding cast. This preserves an explicit proof object.

By contrast, using p to declare $C(x)$ and $C(y)$ interchangeable by kernel conversion without a transport is an equality-reflection principle. Calling it a “certified conversion cache” does not remove that distinction. A safe cache may retain the equality proof and the already checked transported term; it cannot silently redefine definitional equality.

The proposal therefore leaves computation and propositional reasoning separate at the kernel boundary while letting the frontend hide routine, proof-backed transports. This is sufficient for the inspected cast, subtype, and observation problems. A more radical foundation should be justified by examples the conservative approach cannot handle acceptably, not by the visual

length of a tactic script.

14.4 The closest existing ideas

Refinement types and behavioral specifications have substantial precedent. Liquid Types combines refinement inference with predicate abstraction and a finite vocabulary of qualifiers; the useful lesson here is to infer within a controlled interface rather than promise arbitrary mathematical consequence closure [25]. F* explicitly distinguishes refinements of values from refinements of computational behavior, including precondition/postcondition-style specifications [26, 27].

Lean already provides the proof-producing host and much of the automation. Its `grind` framework has cooperating fact-propagation and theory-solving mechanisms, and its documented symbolic-simulation work supports verification-condition generation [22, 24]. Tephra should attach mathematical request and observation interfaces to those capabilities where possible, rather than build a second general-purpose theorem prover.

The distinctive proposal is their application to mathematical object identity, scoped domains, finite observations, selected structures, and the exact behavioral-synthesis boundary found in these repositories. None of these comparisons establishes novelty over the whole literature, and none licenses a productivity claim before implementation.

15 Elaborator architecture and implementation cuts

15.1 The minimum state beyond Lean

Most logical state should remain in Lean. Local parameters are Lean variables; assumptions are Lean proofs; structures are explicit terms or selected local instances; definitions and theorem applications are ordinary declarations. Tephra adds only information needed for operations not already exposed conveniently by those mechanisms:

- a property-rule index with theorem references, matching patterns, roles, and invocation policy;
- source-to-obligation and source-to-evidence records, including exact origins and selected observation routes;
- retained computation certificates or accepted terms and their publication policy;
- a readable projection of the exact statement, structures, open conditions, and accepted result relation.

This allocation follows the syntheses’ warning against inventing a duplicate “workspace” whose main content is already present in Lean’s context [1, 2]. The added index is an acceleration structure and explanation layer, not a source of axioms.

15.2 A rule registration has a theorem, not a separate logic

A registration points to an ordinary Lean theorem and adds nonlogical metadata: principal objects, semantic role names for premises, matching patterns, safe default orientations, applicable observation kinds, and search budgets. The theorem’s actual dependent type is authoritative.

A derivative-transfer registration might identify the premises as *domain open*, *point in domain*, and *identity on domain*. A torsion-preservation registration identifies *chosen additive action* and *torsion witness*. Role labels improve diagnostics and maintain source correspondence when library parameter order changes; they do not override the theorem’s binder dependencies.

The registry validator checks that a metadata role points to the intended binder and that the registered theorem has the advertised result head. A changed theorem signature invalidates the registration until its adapter is updated. Importing a similarly named theorem with a different

contract cannot silently repair the registration.

15.3 Forward facts and backward demands

The resolver should combine two bounded operations. Forward propagation derives cheap, frequently needed consequences, such as $p \geq 5 \Rightarrow p \neq 0$, from already present facts. Backward checking starts with a demanded predicate, finds registered theorem conclusions that match it, and turns their premises into scoped subgoals.

Use a restricted deterministic phase first: direct hypotheses, definitional normalization, selected implication rules, and small arithmetic discharge. More expensive theorem search or Leant composition follows under an explicit budget. Avoid an unrestricted saturation loop that generates all imaginable properties of every term. Cache proved conclusions by the actual object and structure context; do not cache the absence of a proof as a permanent mathematical fact.

The user can inspect a failed demand as a small proof tree. For example:

```
Needed: IsUnit (1 + W x), in the selected real field
Available: x belongs to U; W x != -1
Route: nonzero_of_ne_neg_one -> unit_of_nonzero
Status: discharged; evidence retained
```

A corresponding request in $\mathbb{Z}$ would not use the field route. A request missing $W(x) \neq -1$ would show an open condition rather than strengthen the theorem.

15.4 Reasons can be requirements, not just hints

The controlled syntax distinguishes a required mathematical method from an optional hint. A step marked by `reindex k = j+i` commits to the reindexing theorem and its map, coverage, and summand-equality premises. A proof obtained by an unrelated theorem cannot silently replace that reason.

This requirement is checked against the elaborated method application in the source-evidence record. Merely finding the method's name somewhere among transitive dependencies is too weak. Conversely, unconstrained by `search` explicitly delegates the local proof method while preserving the statement.

Every authored formal assertion is checked, including one unused by the final theorem. A valid final result is not permission to display a false intermediate claim. Pure explanatory prose may be distinguished from formal assertions, but the distinction must be unambiguous to a reader; declarative `have`, `hence`, and calculation lines are always claims.

15.5 Transactional acceptance and publication

A worker result returns to the exact request origin, including its telescope, goal, statement-level assignments, structure footprint, and selected policy. If an edit or undo changed that origin, the result is stale. It may be retained as a candidate for a later rechecked adapter, but it cannot mutate the current proof state directly.

Publication checks several independent conditions: all formal claims have proofs, the top-level theorem matches the fixed statement, the retained method records support the required reasons, the displayed guarantee is no stronger than the checked one, and the actual axiom dependencies satisfy policy. A conditional theorem can be fully checked while an unconditional parent request remains open. The interface should report both facts instead of compressing them into one ambiguous success flag.

Toolchain migration requires replay in the destination environment and a new receipt. A previous axiom inventory, serialized expression, or hash is not a proof under a changed toolchain. Fine-grained reuse after unrelated edits is an optimization to add only after conservative invalidation is reliable.

15.6 A staged implementation with explicit exit conditions

Cut	Implemented artifact	Exit condition
0	Ordinary Lean result/refinement interfaces; one polynomial checker’s denotation, soundness, and source adapter	End-to-end checked positive example and corrupted/arity-mismatch negative neighbors
1	Local property index and property-implicit call elaboration, with no new proof grammar	Exact original terms are reused; missing guards stay open; scope mutations fail
2	Prefix-observation and invariant-subtype adapters	The two worked developments compile; shift and non-subgroup mutations are rejected for the right premises
3	Leant law registration and counterexample realization adapter	Proved provider laws or directly checked source witnesses; <code>List Empty</code> is not falsely rejected
4	Compact authoring syntax and document projection	Same-library comparison shows an improvement beyond what library packaging and display folding already provide

The first cut is deliberately not another expansive grammar. The present article supplies designs and a small reference encoding, not completion of these cuts. The earliest decisive experiment is whether proof-linked property insertion reduces actual author work when ordinary Lean is given the same lemmas and solvers.

16 Executable evidence supplied with this article

16.1 What was run

The archive contains `tephra_checks.py`, a dependency-free Python 3.10+ program. It was executed under Python 3.13.5. All **23 test methods passed**, with zero failures and zero errors. The machine-readable result and the complete test output are included.

The tests include a sparse integer-polynomial certificate with exact coefficient arithmetic, malformed and corrupted certificates, a Catalan residual, derivative precision loss, origin and scope checks, dependency-cycle rejection, a closed list-expression evaluator with a length abstraction, and explicit negative examples for totalized division, completeness, and one-sided stream inverses. One additional test contrasts the abstract positive-length witness with the realizable inputs of lists over an empty element type.

Within the named tests, the multivariate identity is checked at 100 deterministic integer valuations in both $\mathbb{Z}$ and $\mathbb{Z}/4\mathbb{Z}$; the list-length evaluator is checked on 64 input-length pairs; and the finite difference/prefix-sum inverse matrices are multiplied in both orders for sizes 1 through 8. These iteration counts are reported separately from the number of test methods. They are not counts of formal proofs.

16.2 What these tests establish, and what they do not

The coefficient checker decides the finite identity of the supplied executable polynomials. The scope model checks the intended bookkeeping rules for its small immutable request representation.

The tiny list evaluator checks a closed syntax containing inputs, empty lists, append, and reverse; unknown provider nodes are rejected rather than assigned guessed laws.

None of this is a Lean kernel implementation. The Python code does not validate Lean expressions, certify arbitrary provider laws, prove the compiler sound, or establish the universal prefix-local inverse theorem. Its finite polynomial evaluations cross-check the implementation; the generic denotation argument is the written proof in Section 11. The polynomial data checker and the request bookkeeping model are not presented as one end-to-end verified bridge from source Lean syntax.

16.3 The supplied Lean reference encoding

`TephraCore.lean` encodes first-class refinements, contract weakening and composition, object-indexed results, dependent guarded outputs, complete solution predicates, commuting-square observations, subtype restriction, and a realized-counterexample rule. Its source contains explicit proof terms and does not introduce `sorry` or new axioms.

A Lean toolchain was not available in the preparation environment, so this file was **not compiled**. It is a small reference encoding for review, not a validated release. In particular, it is not the polynomial normalization proof or a formalization of Theorem 8.4. Those remain named implementation tasks rather than accomplishments inferred from the presence of a `.lean` file.

16.4 A paired negative suite for the next implementation

The supplied Python tests exercise only a subset of the following obligations. The remaining rows are explicit proposed Lean tests; the table is not a claim that a complete frontend suite has been executed.

Boundary	Positive neighbor	Negative neighbor and required result
Refinement implication	Positive real used as nonzero	Nonnegative real used as nonzero: request missing strictness
Algebraic structure	Unit permits inversion	Regular integer 2 used as a unit: reject the missing unit proof
Locality	Equality on open U differentiated at $x \in U$	Equality at x alone: local-identity obligation stays open
Prefix observation	Prefix-local linear maps restricted to R^N	Left shift restricted at the same N : locality fails
Finite algebra	Square matrices over a commutative ring	Arbitrary infinite endomorphisms: no inverse-reversal adapter
Subtype structure	Torsion subgroup inherits its fixed action	Invariant set omitting zero: no additive-group structure
Chosen action	Exact ambient action restricted	A second action with the same carrier name: require an explicit comparison
Jet precision	$N + 1$ input coefficients for N derivative coefficients	Keep order N after differentiating an N -jet: reject or recompute
Regular branch	Unit derivative and matching constant term	Nonunit derivative, or wrong root constant: cannot identify the jet
Polynomial witness	Exact integer coefficient identity	Corrupt a coefficient or truncate mismatched lists: reject
Coefficient domain	Integer ideal witness	Rational multiplier $1/2$ offered as integer data: reject
Complete solving	Soundness and coverage proved	Witness or coverage alone: retain weaker result only

Boundary	Positive neighbor	Negative neighbor and required result
Behavioral abstraction	Proved length transfer law	Passive provider assertion used as a theorem: no proof authority
Counterexample	Concrete nonempty input of the fixed source type	Positive length for <code>List Empty</code> : unrealizable, not a refutation
Uniformity	Explicit naturality proof	Arbitrary polymorphic Lean function: no automatic free theorem
Scope	Parent fact reused in a child	Sibling or exited-branch fact reused: reject context map
Origin	Result returns to unchanged goal	Undo, changed object, or changed structure: mark stale
Guard dependency	Guard proved independently before its use	Result used to justify its own guard: reject circular assembly
Method fidelity	Stated reindexing theorem actually applied	Different proof offered for a required reindexing step: method mismatch
Publication	All authored assertions checked	False unused assertion beside a true final theorem: document rejected

A test suite should verify not just rejection, but the reason for rejection and the availability of its positive neighbor. Rejecting all computations or all implicit refinements is not a successful implementation.

17 Evaluation: separate mathematical abstraction from language benefit

17.1 Three implementation treatments and a separate reading study

Use three authoring treatments over the same selected task families. The first is idiomatic Lean with all newly developed mathematical lemmas, the same external providers, and normal automation. The second adds proof-linked refinement inference and retained requests but keeps essentially Lean authoring syntax. The third adds the compact mathematical surface. Comparing the first two measures the inference layer; comparing the second and third measures authoring syntax rather than newly privileged library facts.

A separate blinded reading study compares rendered views. Before exposing proof expansion, ask readers to reconstruct the domain, branch, quantifier scope, precision, completeness, and assumptions of each result. Then reveal the expansion and measure whether errors are corrected. This implements the syntheses' most useful presentation test without conflating reading comprehension with proof-construction speed [1, 2].

The registered methods and their supporting theorems must be available to the Lean control. It is invalid to compare a Tephra application of a newly proved general theorem with a Lean proof forced to rederive that theorem from scratch.

17.2 Workloads that distinguish the components

Use four kinds of development tasks: guarded algebra and residual jets; local analysis; finite-observation linear algebra; and subtype/structure constructions. The Leant-facing tasks should include actual proof compositions, not only single-lemma retrieval: establish a unit from a branch condition, propagate a range condition through composition, assemble a restricted action, and justify a realized length counterexample from an exact candidate.

The existing binomial and autonomous-derivative files serve as controls. The prefix-local and torsion developments are design material. Held-out tasks must be chosen separately after the interfaces are fixed, excluding aliases, near duplicates, and downstream results that reveal the target. An uncited filename alone does not establish independence.

17.3 What to measure

Measure author edits, explicit auxiliary proofs, failed inference attempts, diagnosis and repair work, initial elaboration time, search-free replay time, peak memory, and retained proof/certificate size. Record property-index and registration costs separately. Charge the mathematical library and adapter implementation to the approach that needs it, and state how that cost is amortized across tasks.

A theorem that acquired an extra assumption, a narrower domain, or a stronger algebraic structure is not a successful solution of the original task. Such changes may be legitimate author decisions, but they must be categorized as statement changes rather than proof improvements. Likewise, hiding a finite precision bound is a comprehension failure, not a token saving.

The stopping rule should be practical: keep the library and proof-index services if they help, and stop expanding the surface language if it produces no benefit beyond those services and a compact display. A useful library-only outcome is not a failed experiment.

18 Responses to the next-round implementation questions

The second synthesis ends with ten concrete questions. The following answers are commitments of this proposal, not claims that all the corresponding code has been built [2].

	Question	Answer in this design
R1	Checker soundness in practice	Section 11 specifies coefficient data, valuation, normalization/evaluation lemmas, certificate theorem, and a separate exact source reification bridge. Complete that bridge in Lean before adding another checker.
R2	Where does <code>grobner</code> sit?	It is an existing proof-producing control and possible provider. Use its accepted term initially. Do not assume a standalone export interface or turn bounded failure into a negation.
R3	What is new workspace state?	The property index, source-evidence mapping, observation-route record, and retained receipt. Mathematical variables, assumptions, and structures remain ordinary Lean objects.
R4	Partial or total by default?	Preserve total semantics for imported Lean expressions. Offer an explicit partial-expression package whose domain is part of the type. No invisible switching.
R5	Which package first?	One end-to-end polynomial certificate and guarded-result interface, followed by the property-implicit binder. Use prefix locality and torsion restriction to test the specific third-round extension.
R6	When is native evaluation worthwhile?	No threshold is claimed. Measure the actual checker and workload after the strict implementation exists, and report its axiom policy independently of speed.
R7	Toolchain upgrades	Recheck in the destination environment, including the exact statement and transitive axiom dependencies, and issue a new receipt. A historical axiom name is not a policy.
R8	What does Leant add?	Structural assembly under proved behavioral constraints, plus candidate and counterexample generation. Evaluate composition obligations and realization, not merely lemma lookup.

	Question	Answer in this design
R9	Can mutations be executed?	Twenty-three finite boundary tests are supplied. The larger paired table is an explicit Lean/frontend test backlog, not an executed suite.
R10	Is another large batch of designs useful?	The next campaign should prioritize independent implementations of the same small interface and an adversarial review. Additional names and syntax variants are less informative than a proved adapter and a measured authoring experiment.

The parallel synthesis’s additional concerns are also concrete here. Delegation is allowed by an explicit specification, not by silently changing fixed data. Domain, branch, precision, completeness, and unresolved conditions remain visible. A stated method can constrain the elaborated proof. Competing representation routes require a recorded choice or a coherence theorem. The same services are available to the baseline, and the reading study determines whether compact rendering communicates the guarantee faithfully.

19 Conclusion

The proposed extension is not “Lean, but trust an AI to fill the gaps.” It is a mathematical type-and-evidence layer whose implicit steps have precise contracts. A property of an existing object remains a proof about that object. A transformation’s behavior is an ordinary quantified proposition. An observation is indexed by its original semantic object and can be transported or reflected only through the appropriate theorem.

Three consequences distinguish this design from another notation proposal. Prefix-locality becomes a usable type-level contract that justifies finite restriction and explains the failure of an infinite inverse argument. Invariant-subtype construction becomes a derived interface for a chosen action rather than repeated coercion and law plumbing. Behavioral synthesis gains a proof-linked law and realization boundary, including the cases where a plausible abstract counterexample cannot exist at the source type.

These improvements do not require adding arbitrary semantic subtyping or equality reflection to Lean’s kernel. They require a disciplined elaborator, theorem-backed rules, and a small amount of retained nonlogical metadata. A genuine native refinement former remains an option if measurements identify a benefit that the conservative encoding cannot provide. The immediate technical objective is smaller: implement one complete checker boundary and one property-aware call interface, preserve the difficult negative examples, and determine whether authors can state their mathematical reasons while the system reconstructs the routine evidence reliably.

A A compact semantic interface

The following excerpt summarizes the interface shape. It is explanatory Lean-like notation; the archive’s separate `TephraCore.lean` contains the fuller reference encoding and its explicit validation status.

```
structure Result (R : A -> B -> Prop) (a : A) where
  value : B
  sound : R a value

def Contract (P : A -> Prop) (Q : A -> B -> Prop)
  (f : A -> B) : Prop :=
  forall x, P x -> Q x (f x)

structure ObservationMap (oa : A -> U) (ob : B -> V)
  (f : A -> B) where
  map : U -> V
  commutes : forall x, ob (f x) = map (oa x)

structure CompleteSolutions (P : A -> Prop) where
  member : A -> Prop
  sound : forall x, member x -> P x
  complete : forall x, P x -> member x
```

The indices are mathematically meaningful. Replacing a by a string name, changing the chosen observation, or dropping the coverage proof changes what the interface guarantees. Production representations may use optimized storage and hashes, but must reconstruct these exact types at acceptance.

The surface needs only a small initial family of constructs: bind objects and assumptions; define or construct by specification; assert and calculate with a named reason; introduce cases, induction, and local domains; request a typed computation or observation; and embed an ordinary Lean proof. A general natural-language parser is not part of the required first implementation.

B Source snapshots and reading coverage

The identifiers below are Git *blob* hashes, not repository commit hashes. They identify the individual retrieved file contents. The accompanying `source_register.json` records full repository paths, default-branch display URLs, blob API locators, retrieval date, and reading coverage. Files were inspected on 6 September 2026. This register does not claim an atomic checkout or a successful build of any repository.

ID	File and exact blob
S1	VladimirReshetnikov/Brainstorm ; docs/round-2/synthesis/unified-report.tex ae7c21dac1ba615e36dff07ea6f6a1c433a40108. Entire TeX source.
S2	VladimirReshetnikov/Brainstorm ; docs/round-2/unified_report/unified_report.tex de10daae20af19285ee07a476caa94e577f7d7d9. Entire TeX source.
P1	VladimirReshetnikov/ProveIt ; Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean 1422b87f31073fed898ff0c8a080ea0a798aef5e. Entire file.
P2	VladimirReshetnikov/ProveIt ; Analysis/FabiusFunction/Lean/FabiusFunction/BinomialInversion.lean c6b0fe63a4faf8708b71e9b93aa2ce7406464d84. Entire file.

ID	File and exact blob
P3	VladimirReshetnikov/ProveIt; Analysis/FabiusFunction/Lean/FabiusFunction/TriangularKernelInverse.lean 8454db52a488823f6314032382c45a357d93a5e4. Entire file.
F1	anthropics/fermats-last-theorem; Definitions/Def_FLTPrelim_FreyPackage.lean db1cddb9c434a118a917b33580fc8c81c4fff221. Entire file.
F2	anthropics/fermats-last-theorem; Definitions/Def_FLTPrelim_GaloisRep.lean 998083d1f0de2d0271a04e7e099cb0e3092d0b3e. Entire file.
F3	anthropics/fermats-last-theorem; P2M/Sol/S_FreyPackage_no_frey_package.lean 285ecf230276505ab9d9bb0ba025665b68196891. Mathematical tail and sampled instance-management preamble.
F4	anthropics/fermats-last-theorem; PROOF-PATH.md 5a5d69b8f946ddc7587e300632cf924d6bf3373c. Entire file.
L1	VladimirReshetnikov/Leant; src/Leant/Synth/Verification.hs caee3c51cfd226e6c2bb9887f80293343abd486c. Entire file.
L2	VladimirReshetnikov/Leant; src/Leant/Synth/Fragment.hs ab523ffd5e93a0ddc8c5d893a79e032562319aa1. Introductory fragment/provenance definitions; not the entire module.
L3	VladimirReshetnikov/Leant; src/Leant/Synth/BehavioralSelection.hs 6887c30133f72d984d3a128e6c8cd37acb37aae3. Entire public boundary module; not all internals.
L4	VladimirReshetnikov/Leant; src/Leant/Synth/Length/Contract.hs a1f04001c6ccd6c5e018162fa4db7ff5b059972e. Entire file.
L5	VladimirReshetnikov/Leant; src/Leant/Synth/Length/Adapter.hs efa9c1835ec109f43b7d7cdf916eec7ab166656e. Entire file.
L6	VladimirReshetnikov/Leant; src/Leant/Synth/Replay.hs 298749c8904678c88c1dcc030eb0ec95e68fdd9c. Entire file.

C Reproducing the delivered artifacts

Build the article with `build.sh`, or run `pdflatex tephra.tex` three times. It uses common TeX packages, Latin Modern fonts, and no external figures or bibliography processor. Run `python tephra_checks.py` to reproduce the executable tests and regenerate `check_results.json`. The Python program uses only the standard library.

The `TephraCore.lean` file requires separate compilation in a Lean 4 environment. No compiled Lean object or kernel-check transcript is included, because none was produced. The source and README distinguish the reference encoding from the implemented Python model and from the proposed production elaborator.

References

- [1] Brainstorm contributors. Round-two synthesis report, parallel synthesis directory. Source S1 in Appendix B. <https://github.com/VladimirReshetnikov/Brainstorm/blob/main/docs/round-2/synthesis/unified-report.tex>.
- [2] Brainstorm contributors. Round-two synthesis report, unified report directory. Source S2 in Appendix B. https://github.com/VladimirReshetnikov/Brainstorm/blob/main/docs/round-2/unified_report/unified_report.tex.
- [3] Vladimir Reshetnikov and contributors. Iterated derivatives of an autonomous equation. Source P1 in Appendix B. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean>.
- [4] Vladimir Reshetnikov and contributors. Binomial inversion. Source P2 in Appendix B. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/Lean/FabiusFunction/BinomialInversion.lean>.
- [5] Vladimir Reshetnikov and contributors. One-sided orthogonality of triangular kernels is two-sided. Source P3 in Appendix B. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Analysis/FabiusFunction/Lean/FabiusFunction/TriangularKernelInverse.lean>.
- [6] FLT contributors. Definition and elementary properties of the Frey package. Source F1 in Appendix B. https://github.com/anthropics/fermats-last-theorem/blob/main/Definitions/Def_FLTPrelim_FreyPackage.lean.
- [7] FLT contributors. Galois action on points and torsion; irreducibility definition. Source F2 in Appendix B. https://github.com/anthropics/fermats-last-theorem/blob/main/Definitions/Def_FLTPrelim_GaloisRep.lean.
- [8] FLT contributors. Solution module for the nonexistence of a Frey package. Source F3 in Appendix B. https://github.com/anthropics/fermats-last-theorem/blob/main/P2M/Sol/S_FreyPackage_no_frey_package.lean.
- [9] FLT contributors. The route of the proof and exact strength of named steps. Source F4 in Appendix B. <https://github.com/anthropics/fermats-last-theorem/blob/main/PROOF-PATH.md>.
- [10] Vladimir Reshetnikov and contributors. Leant candidate-verification receipts. Source L1 in Appendix B. <https://github.com/VladimirReshetnikov/Leant/blob/main/src/Leant/Synth/Verification.hs>.
- [11] Vladimir Reshetnikov and contributors. Leant synthesis-fragment translation and provenance. Source L2 in Appendix B. <https://github.com/VladimirReshetnikov/Leant/blob/main/src/Leant/Synth/Fragment.hs>.
- [12] Vladimir Reshetnikov and contributors. Leant behavioral-selection public boundary. Source L3 in Appendix B. <https://github.com/VladimirReshetnikov/Leant/blob/main/src/Leant/Synth/BehavioralSelection.hs>.
- [13] Vladimir Reshetnikov and contributors. Leant finite-spine length-contract vocabulary. Source L4 in Appendix B. <https://github.com/VladimirReshetnikov/Leant/blob/main/src/Leant/Synth/Length/Contract.hs>.
- [14] Vladimir Reshetnikov and contributors. Leant checked length-query adapter. Source L5 in Appendix B. <https://github.com/VladimirReshetnikov/Leant/blob/main/src/Leant/Synth/Length/Adapter.hs>.
- [15] Vladimir Reshetnikov and contributors. Leant environment replay planning. Source L6 in Appendix B. <https://github.com/VladimirReshetnikov/Leant/blob/main/src/Leant/Synth/Replay.hs>.
- [16] Lean Language Reference. Subtypes. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/Basic-Types/Subtypes/>.

- [17] Lean Language Reference. Inductive types, including proposition elimination and runtime representation. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/The-Type-System/Inductive-Types/>.
- [18] Lean Language Reference. Type classes. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/Type-Classes/>.
- [19] Lean Language Reference. Validating a Lean proof. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [20] Lean Language Reference. Axioms, including the parametricity counterexample. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/Axioms/>.
- [21] Lean Language Reference. Tactic reference, SMT-inspired automation. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [22] Lean Language Reference. The grind tactic. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [23] Lean Language Reference. Algebraic solver for commutative rings and fields. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/The--grind--tactic/Algebraic-Solver-LPAR-Commutative-Rings---Fields-RPAR/>.
- [24] Lean Language Reference. Lean 4.28.0 release notes: symbolic simulation framework. Accessed 6 September 2026. <https://lean-lang.org/doc/reference/latest/releases/v4.28.0/>.
- [25] Patrick Rondon, Ming Kawaguchi, and Ranjit Jhala. Liquid Types. PLDI 2008, pp. 159–169. Author publication page. Accessed 6 September 2026. https://goto.ucsd.edu/~rjhala/papers/liquid_types.html.
- [26] F* team. Proof-Oriented Programming in F*: Getting off the ground, refinement types. Accessed 6 September 2026. https://fstar-lang.org/tutorial/book/part1/part1_getting_off_the_ground.html.
- [27] F* team. Proof-Oriented Programming in F*: Primitive effect refinements. Accessed 6 September 2026. https://fstar-lang.org/tutorial/book/part4/part4_pure.html.